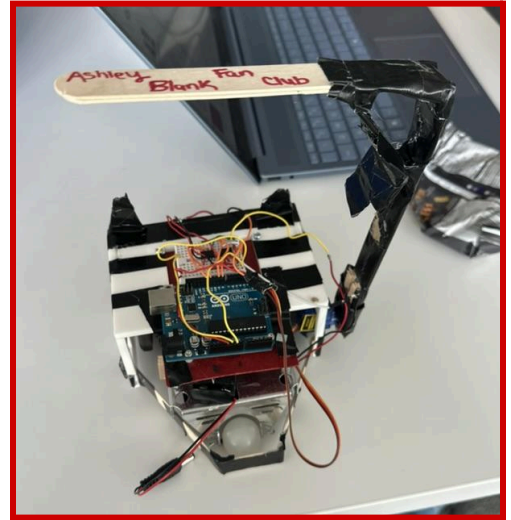


Team 52 Final Report: Ashley Blank Fan Club

Ava Farkash (ajf245), Alex Weng (akw76), Abigail Miller (aem337)

Robot Design and Strategy Overview

For the mechanical design of our robot, we wanted to design an arm that can scoop up blocks and hold them while both turning and moving forward. To do this, we first designed a laser-cut acrylic frame to protect the wheels, then used tongue depressors and multiple adhesives for the L-arm. The robot uses two servos to drive each wheel and one continuous micro servo to control the L-arm. We opted not to use QTI, color, or sonar sensors in our final design. This choice was made to prevent errors with timing interrupts and faulty readings that would cause inconsistent performance. Our final code defines a drive system allowing the robot to move using motor control pins on PORTB. It then configures Timer2 interrupts to generate a repeating 20 ms servo control signal on PD2, which controls the micro servo. The servos for the wheels are powered by a 6V battery pack, and the Arduino is powered by a 9V battery. During competition, the robot runs the main function, which initializes the motor outputs and the servo system and deploys the L-arm from vertical to touch the board. The robot then drives forward two feet, executes the spiral collection loop six times, performs one final collection motion, and stops completely. The spiral collection loop has the robot move forward one foot, then open the arm 30 degrees, and move forward to collect any blocks pushed in front of it. After the arm has lowered again, the robot turns 210° to the left and accumulates blocks in the crux of the “L”. The blocks in the L-Arm are then kept inside the robot's perimeter and protected from being stolen.



Design Process Reflection

We began the design process with a group discussion about the proposed strategy. From the first meeting, we decided the robot should focus on block collection rather than defence or evasion, and that a simple design was best. Brief sketches of possible ideas were drawn and critiqued based on questions from the group members. The 210° rotating T-arm idea was created in the second group meeting as an effective, easily manufacturable, and budget-conscious subsystem. The “scorpion legs” (tongue depressors sticking directly out from the chassis) were also suggested at this stage as walls to contain the blocks at the front and back of the robot. As Milestone One (M1) approached, Ava finalized the initial sketch as part of the design overview, Abby created the proposed strategy flowchart, and Alex wrote our design motivation.

After the successful presentation of M1, we began working on the mobility code for Milestone Two (M2) and brainstorming materials for the T-arm and wheel protection casing. Upon reading the RPL 3D print and laser cut pricing, we agreed that it would be a good use of our budget to laser cut acrylic for the protection casing. At this point, we decided to use lightweight wood for our T-arm with a 3D-printed joint. We also wired the two servo motors for the wheels using two L9110H H-bridges, and worked together in

the lab to test the delay time to move and turn the distances required. In the next lab session, we attempted M2 and were slightly off the projected path. Thus, Abby edited the code, we tested again, and passed M2. We noticed the wheels were slippery on the mat during M2, so we added the provided elastic covering to resolve this issue.

As Milestone Three (M3) approached, we integrated a color sensor on the bottom of our robot. We decided against adding a sonar or QTI sensor and instead used the color sensor to distinguish between the three colors on the board. At this stage, Abby worked on the code, Alex designed the wheel protection casing on CAD, and Ava worked on the logistics of arm integration and materials. We submitted a request for tongue depressors from McMaster in part order three to use for our T-arm and scorpion legs. We decided to change the T-arm to an L-arm because we had too many components (the Arduino, breadboard, wires) on top of our robot that made mounting the servo difficult. By deciding to install the L-arm on the side of the robot, we avoided the circuitry and retained the desired range of motion. To accomplish M3, we formulated a code that could determine which color the robot was driving on. To do this, we tested the period reading for each color and set appropriate thresholds. We ran into a challenge while testing because the code successfully worked when the robot was plugged into the computer, but not the 9V battery. Eventually, we realized our Arduino was only reading a voltage of 2.5V instead of 5V. This was fixed by changing the 9V battery, and we successfully were able to detect the centerline, turn around, and stop at the edge of the board to accomplish M3.

En route to Milestone Four (M4), Ava prototyped the L-arm, Alex attached the wheel protection casing, and Abby wrote and tested the cube-clearing code. The L-arm was created by hot gluing and taping tongue depressors together while constantly referencing the constraints to ensure compliance with the eight-inch rule. The scorpion legs were integrated and secured through slots in the wheel protection casing, again keeping in mind the length constraints. In the process of attaching the scorpion legs, we encountered a problem: the reach would be too short on each side, restricting the number of blocks the robot could collect. After a group debrief, we decided to remove the back scorpion arms and extend the front ones. This also meant we changed our strategy. Instead of switching the front of our robot depending on the direction we were driving and the position of the L-arm, we decided to implement a turning algorithm that collected blocks in the L-arm as we rotated in a spiral. This allowed us to complete M4.

Leading up to the competition, we attempted to add a second L-arm to the other side of the robot using an additional micro servo. However, when attempting to code the servos to work together simultaneously, we encountered issues getting the second servo to rotate. After a couple of hours of debugging, we decided to revert to the singular L-arm used in M4. Ava created a second iteration of the L-arm with a cardboard brace to hold the 90° angle of the "L" and duct taped/superglued the two tongue depressor layers together to increase the stiffness of the arm. The L-arm was then secured to the servo by sliding the horn between tongue depressor guide rails and using glue and duct tape to strengthen the attachment. The final agreed-upon strategy was to start facing forward and quickly move to the center of the board. The robot then spins 270 degrees to the left while collecting blocks in the L-arm. After the arm is shut again, the robot moves forward one foot and lifts the arm 30 degrees to collect any blocks it had pushed. It then repeats this sequence six times.

Competition Analysis

During the competition, our robot started out strong with a win. During the first round, the other robot did not collide with ours, and we were able to collect and protect a number of blocks using the spiral loop algorithm. After this, our robot lost the other rounds as the arm got stuck in the other robots. We realized that although the robot arm and algorithm were good at collecting blocks without obstacles/an opponent, it was not able to untangle itself from other robots in the case of a collision and wasn't effective during the competition. When the arm was sweeping up blocks, if there was a robot to our left, the arm would get stuck around the other robot. Since our robot only turned to the left, it was never able to move away. Additionally, our robot was relatively small and lightweight, so some of the other robots were able to push it off the board. The last challenge we encountered was during one of the later rounds, when the power wire for the micro servo controlling the L-arm came unplugged. Because the arm never rotated down at the beginning, and we had no collection method. We were able to fix this for the following rounds. Overall, our algorithm worked properly during the competition, but we ran into some challenges we had not accounted for.

Conclusions

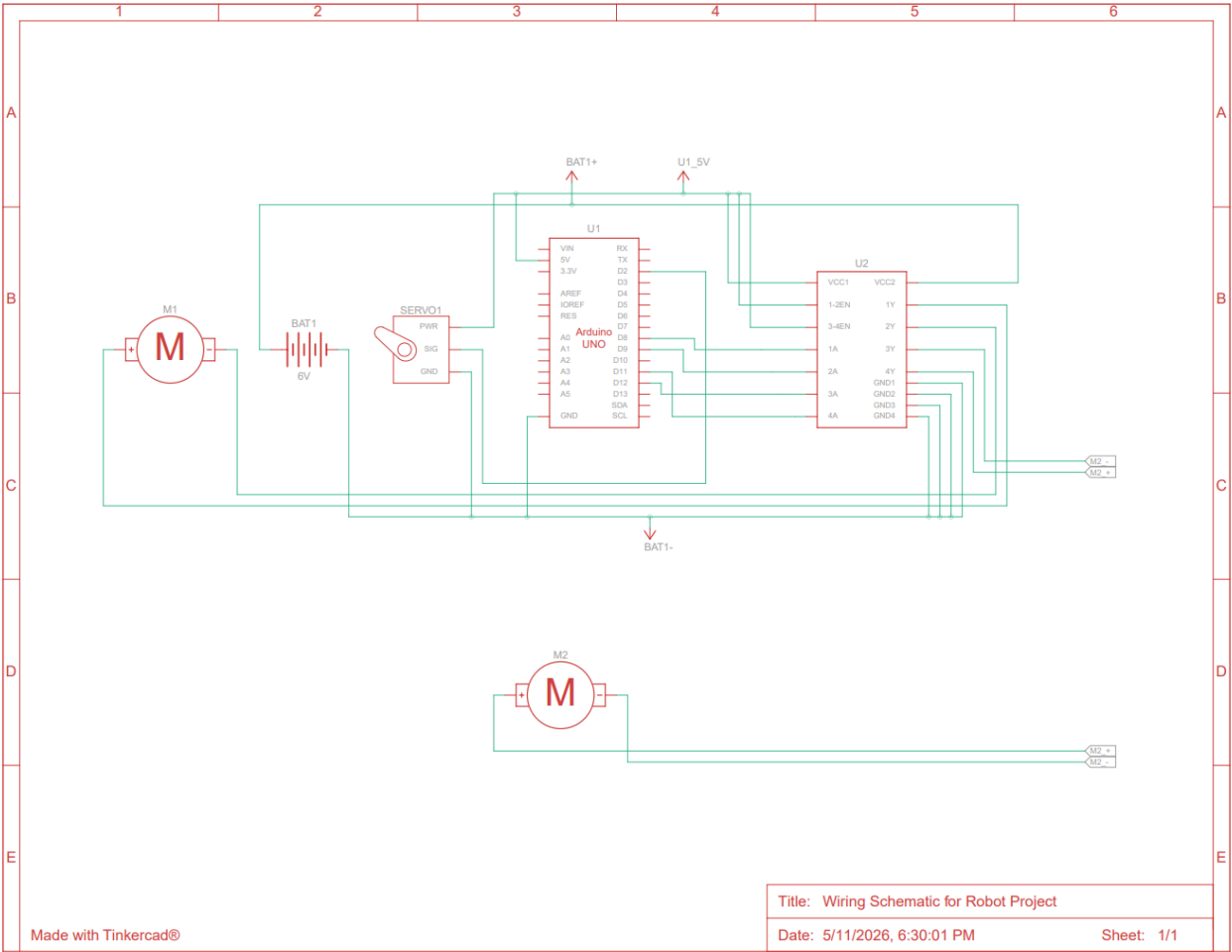
There are many things we would do differently given the opportunity to redo the competition. First of all, we greatly underestimated how much the presence of another robot would disrupt our own robot. We didn't anticipate the amount of contact and collisions that happened on the game board, and as a result, our robot only got to complete its hard-coded movements one time uninhibited. That also happened to be the only round that we won. If we were to redesign, we would make the robot heavier so it would be more difficult to push around and could push other robots in a collision. Next, we would also choose to use sturdier materials if we kept the L arm design; although the popsicle sticks were sturdy in the sense that they didn't break, the L arm bent during collisions, and other robots were able to steal our blocks. Lastly, we would use at least one color sensor in the future, because contact with other robots inevitably causes our robot to deviate from its original path. This means that it may run off the board, even if it didn't do so when tested alone. If we include one color sensor at the front of the robot, it could detect when we hit the black border and interrupt the spiral loop, so that the robot wouldn't continue and accidentally drive over the edge. We could also use tape around the color sensor to block outside light that makes the sensor readings inaccurate.

Another thing we would change is to use more of our budget. We overestimated the cost of the laser cuts, so we didn't investigate other materials we could buy. We definitely could have invested in bigger wheels or 3D printed the L arm. However, we agree that we had overall good design choices and that sticking to a simpler design was better. Many teams that did well in the competition (including the winning team) did not use any sensors, just like us. Additionally, we witnessed that many teams with complicated designs (such as multiple sensors or 4+ servos) ended up having malfunctions that prevented them from doing well. Because of this, we would advise future competitors to also stick to a simple yet effective design that can compete with other robots in the event of a collision.

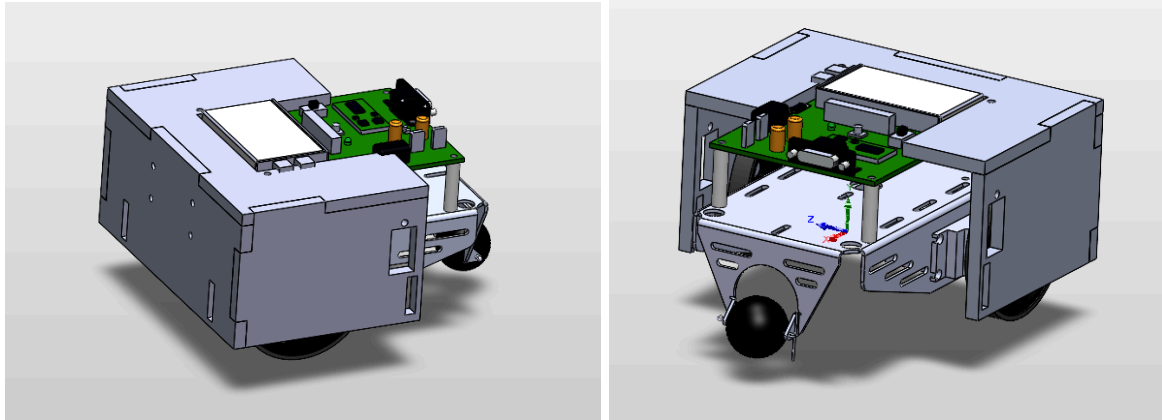
Appendix A: Bill of Materials

Material	Intended Use	Company	Part Name	Unit Cost	Quantity	Total
Balsa	L-arm & scorpion legs	McMaster	Tongue Depressors	\$7.31	1 Box (100 count)	\$7.31
White Acrylic	Laser cutting material	RPL	1 foot by 1 foot white acrylic sheet	5.00	1	\$5.00
White Acrylic	Wheel protection casing	RPL Laser Cut	Front Piece	1.65	1	\$1.65
White Acrylic	Wheel protection casing	RPL Laser Cut	Side Piece	1.40	2	\$2.80
White Acrylic	Wheel protection casing	RPL Laser Cut	Top Piece	1.80	1	\$1.80
					Total:	\$18.56

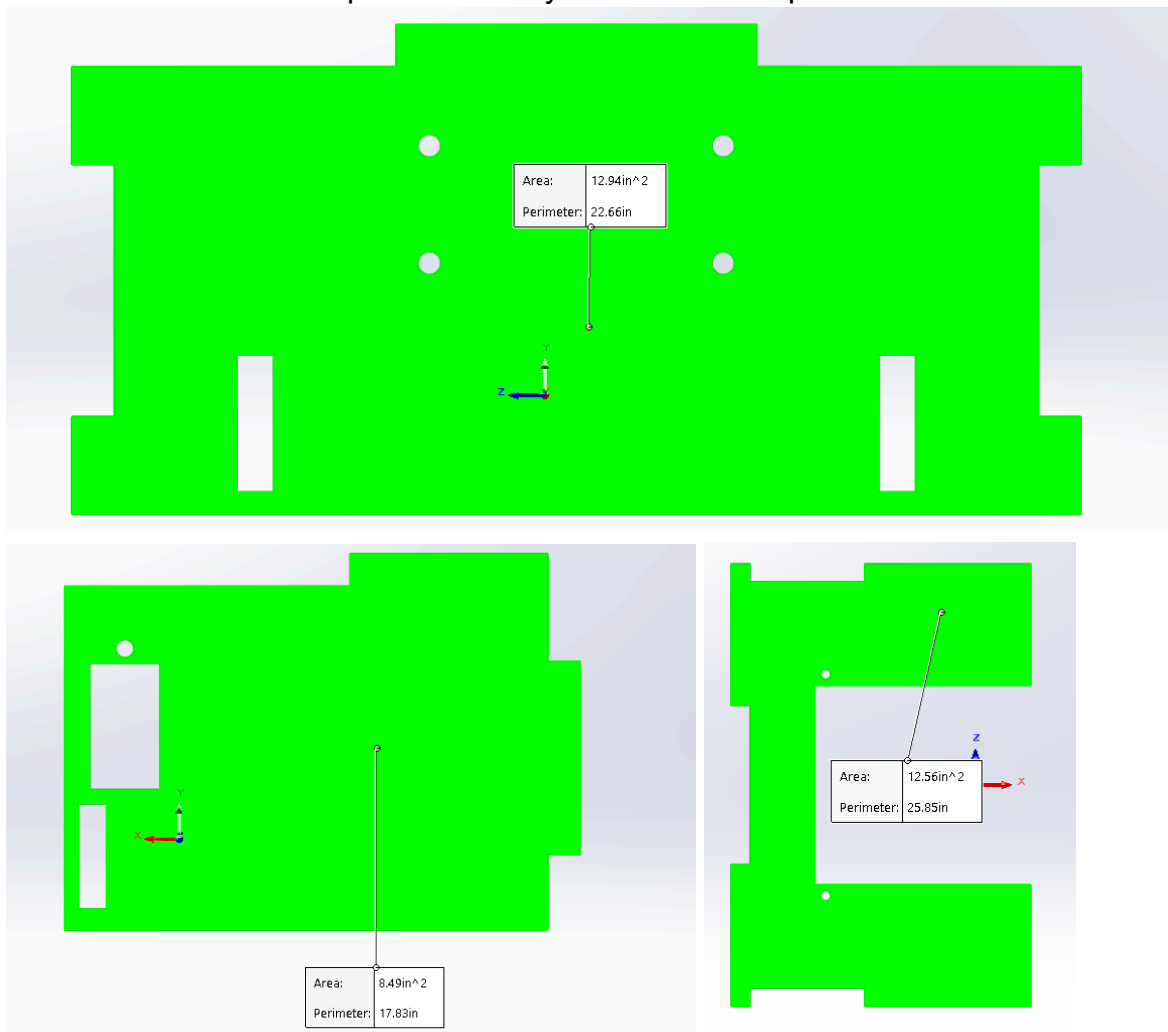
Appendix B: Circuit Diagram



Appendix C: CAD screenshots

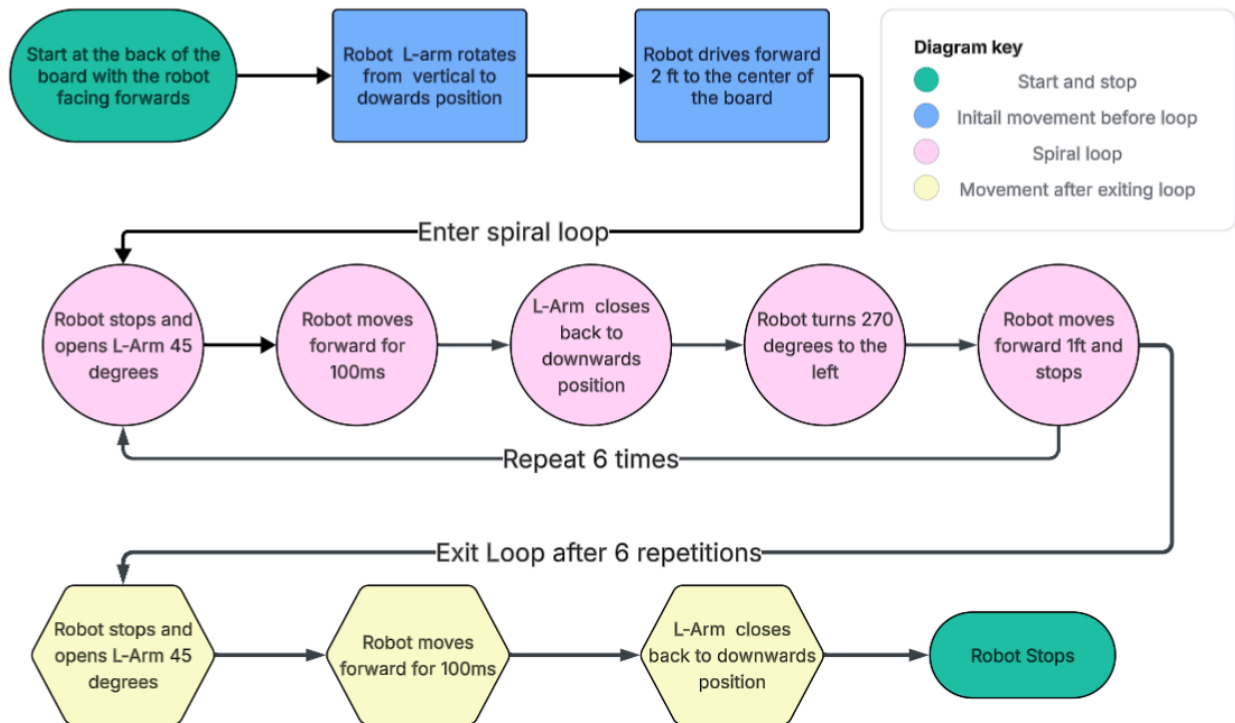


A screenshot of the complete assembly of the laser cut parts on the robot chassis.



Screenshots of the three different .dxf shapes that were printed. They show that the perimeters (cut lengths) of the front, sides, and top pieces are 23 inches, 18 inches, and 26 inches, respectively.

Appendix D: Strategy Flowchart



Appendix E: Code

```
1 //Reference ChatGPT for timer2 interrupts and for initializing the servo and setting up the servo API
2
3 // Global Variables for timing
4 volatile unsigned int period = 0;
5 volatile unsigned int timer = 0;
6
7 // DRIVE SYSTEM (controlled motor movement using PORTB)
8 //stops both motors
9 void stop(void)
10 {
11     PORTB = (PORTB & 0b00000100);
12     _delay_ms(80);
13 }
14 // Drive robot forward
15 // PB4 HIGH = left wheel forward
16 // PB1 HIGH = right wheel forward
17 void driveForward(void) { PORTB = (PORTB & 0b00000100) | (0b00010010 & 0b00011011); }
18
19 // Drive robot backward
20 // PB3 HIGH = left wheel backward
21 // PB0 HIGH = right wheel backward
22 void driveBackward(void) { PORTB = (PORTB & 0b00000100) | (0b00001001 & 0b00011011); }
23
24 // Move robot forward approximately 1 foot (drive forward for 1800ms)
25 void forward_1ft(void)
26 {
27     PORTB = (PORTB & 0b00000100) | (0b00010010 & 0b00011011);
28     _delay_ms(1800);
29 }
30
31 // Turn robot left approximately 90 degrees
32 // PB3 HIGH = left wheel backward
33 // PB1 HIGH = right wheel forward
34 void turnLeft_90(void)
35 {
36     PORTB = (PORTB & 0b00000100) | (0b00001010 & 0b00011011);
37     _delay_ms(600);
38 }
39
40 // Turn robot right approximately 90 degrees
41 // PB4 HIGH = left wheel forward
42 // PB0 HIGH = right wheel backward
43 void turnRight_90(void)
44 {
45     PORTB = (PORTB & 0b00000100) | (0b00010001 & 0b00011011);
46     _delay_ms(550);
47 }
48
49 // Small left turn (~30 degrees- 250ms)
50 void turnLeft_30(void)
51 {
52     PORTB = (PORTB & 0b00000100) | (0b00001010 & 0b00011011);
53     _delay_ms(250);
54 }
55
56 // Small right turn (~30 degrees- 250ms)
57 void turnRight_30(void)
58 {
59     PORTB = (PORTB & 0b00000100) | (0b00010001 & 0b00011011);
60     _delay_ms(250);
61 }
62
63
64 // TIMER2 SERVO SYSTEM
65 // PD2 controls servo
66 volatile uint16_t servo1Pulse = 1500; // Servo pulse width in microseconds
67 volatile uint16_t msTick = 0; // Millisecond counter for servo frame timing
68 volatile uint8_t servo1State = 0; // Tracks whether servo signal is HIGH or LOW
69
70 void timer2_init(void) // Configures Timer2 for 1ms interrupts
71 {
72     TCCR2A = (1 << WGM21); // CTC mode
73     TCCR2B = (1 << CS22); // prescaler 64
74     OCR2A = 249; // 1ms tick
75     TIMSK2 = (1 << OCIE2A); // Enable compare interrupt
76 }
77
```



```

78 // TIMER2 INTERRUPT
79 ISR(TIMER2_COMPA_vect)
80 {
81     msTick++; // Increment millisecond timer
82
83     // Start of 20ms servo frame
84     if (msTick == 1)
85     {
86         PORTD |= (1 << PD2); // Servo pin HIGH
87
88         servo1State = 1;
89     }
90
91     // Keep signal HIGH for pulse duration
92     if (servo1State)
93     {
94         if (msTick >= (servo1Pulse / 1000)) // Convert microseconds to milliseconds
95         {
96             PORTD &= ~(1 << PD2); // Set servo pin LOW
97             servo1State = 0;
98         }
99     }
100
101     // Reset every 20ms
102     if (msTick >= 20)
103     {
104         msTick = 0;
105     }
106 }
107
108 // SERVO INIT
109 void initServos(void)
110 {
111     DDRD |= (1 << PD2); // Configure PD2 as output
112     timer2_init(); // Initialize Timer2
113     servo1Pulse = 1500; // Default servo state = stop
114 }
115
116
117
118 // SERVO API
119 void setServo1(uint16_t pulse) // Sets servo pulse width
120 {
121     servo1Pulse = pulse;
122 }
123
124 // SERVO CONTROL FUNCTIONS
125 // Continuous rotation servo control
126 // Typical values:
127 // 1000 µs = full CW
128 // 1500 µs = stop
129 // 2000 µs = full CCW
130 void servo1_stop(void) { setServo1(100); }
131 void servo1_fullCW(void) { setServo1(2000); }
132 void servo1_fullCCW(void) { setServo1(3000); }
133
134 // Servo MOVEMENTS
135 void servo1Spin_CW_90(void) // Rotate servo clockwise 90 degrees
136 {
137     servo1_stop();
138     _delay_ms(50);
139
140     servo1_fullCW();
141     _delay_ms(500);
142
143     servo1_stop();
144     _delay_ms(50);
145 }
146
147
148 void servo1Spin_CCW_90(void) // Rotate servo counterclockwise 90 degrees
149 {
150     servo1_stop();
151
152     _delay_ms(50);
153
154     servo1_fullCCW();
155     _delay_ms(500);
156
157     servo1_stop();
158     _delay_ms(50);
159 }

```

```

160
161 void servo1Spin_CW_loop(void) // Small clockwise servo movement for the spiral loop
162 {
163     servo1_stop();
164     _delay_ms(50);
165
166     servo1_fullCW();
167     _delay_ms(300);
168
169     servo1_stop();
170     _delay_ms(50);
171 }
172
173 void servo1Spin_CCW_loop(void) // Small counterclockwise servo movement for the spiral loop
174 {
175     servo1_stop();
176     _delay_ms(50);
177
178     servo1_fullCCW();
179     _delay_ms(300);
180
181     servo1_stop();
182     _delay_ms(50);
183 }
184
185 // SPIRAL MOVEMENT ROUTINE for main function (repeats 6 times)
186 void spiral(void)
187 {
188     int counter = 0;
189     while(counter < 6){
190         // Lift the sevro arm slightly
191         // Move forward for 100ms
192         // Arm closes again to collect the blocks
193         stop();
194         servo1Spin_CCW_loop();
195         driveForward();
196         _delay_ms(100);
197         servo1Spin_CW_loop();
198
199         // Turn 270 degrees
200         turnLeft_90();
201         turnLeft_90();
202         turnLeft_90();
203
204         // Drive forward 1ft
205         forward_1ft();
206         stop();
207
208         // Increment loop counter
209         counter++;
210     }
211 }
212
213

```

```

214 // MAIN Function
215 int main(void)
216 {
217
218     DDRB |= 0b00011011;    // Configure motor pins as outputs
219     Serial.begin(9600);    // Initialize serial communication
220     cli();                 // Disable interrupts during setup
221     initServos();          // Initialize servo system
222     sei();                 // Enable interrupts
223
224     servo1Spin_CW_90();    // Initial servo movement to put servo arm down
225
226     forward_1ft();         // Drive forward 2ft
227     forward_1ft();
228
229     spiral();              // Execute spiral movement routine
230     _delay_ms(10);
231
232     // Final movement sequence:
233     // Lift the sevro arm slightly
234     // Move forward for 100ms
235     // Arm closes again to collect the blocks
236     stop();
237     servo1Spin_CCW_loop();
238     driveForward();
239     _delay_ms(100);
240     servo1Spin_CW_loop();
241
242     // Stop robot
243     stop();
244 }

```